

DBMS — Unit 1: Introduction to Database Management Systems

RV University | School of Computer Science and Engineering

1. Historical Context — Edgar F. Codd

- **Edgar Frank "Ted" Codd (1923–2003)** — IBM researcher; universally recognized as the **Father of the Relational Database Model**.
 - In **1970**, published the landmark paper: "*A Relational Model of Data for Large Shared Data Banks*" — [Full paper \(UPenn archive\)](#)
 - **Key contributions:**
 - Introduced the **relational model** — representing data as mathematical *relations* (tables).
 - Formalized **relational algebra** and **relational calculus** as query foundations.
 - Defined **12 Rules** (Codd's Rules) for what constitutes a fully relational DBMS.
 - Eliminated the need for programmers to know physical storage layout — a paradigm shift from hierarchical and network models.
 - **Impact:** Every modern RDBMS (MySQL, PostgreSQL, Oracle, SQL Server) is built on Codd's foundational theory.
-

2. File System vs. Database System

2.1 File System — Deep Dive

Definition: A File System is a method and data structure that an operating system uses to control how data is **stored, accessed, and organized** on a storage medium (hard disk, SSD, etc.) as a collection of named files.

- Data is stored in **discrete, independent files** — each file is owned and interpreted by its creating application.
- The **OS manages** file creation, deletion, reading, and writing at the byte/block level.
- There is **no global schema** — each application defines its own internal record structure.
- **No centralized dictionary** of what data exists where.

Characteristics of a File System

Property	Behavior
Data Storage Unit	Individual files (.txt, .csv, .xml, .json, .docx, etc.)
Data Management	Handled by the Operating System
Data Format	Defined per application — no standard
Data Control	Application-dependent; no global authority
Query Mechanism	No native query language; custom code required
Concurrency	Primitive; multiple writers cause corruption
Security	File-level OS permissions only (read/write/execute)
Redundancy	Uncontrolled; same data duplicated across files
Integrity Enforcement	None — application must implement manually
Crash Recovery	Manual backup; no automatic recovery

Common Examples

- Storage media: hard disks, pen drives, SSDs
- File types: music (.mp3), videos (.mp4), documents (.docx), structured data (.xml, .json), spreadsheets (.xlsx)

2.2 Database System — Deep Dive

Definition: A Database System stores data in a **structured, organized, and interrelated** manner, managed by a **Database Management System (DBMS)** — a software layer that provides centralized control over data definition, access, security, and integrity.

- Data organized in **tables (relations)** with defined schemas and inter-table **relationships**.
- The DBMS acts as an **intermediary** between users/applications and the physical data store.
- Provides a **unified interface** (SQL) for all data access operations.

Characteristics of a Database System

Property	Behavior
Data Storage Unit	Tables (relations) with rows and columns
Data Management	DBMS-controlled
Data Format	Strictly defined by schema
Data Control	Centralized (DBMS enforces all rules)
Query Mechanism	SQL — structured, declarative, powerful
Concurrency	Concurrency control mechanisms (locking, MVCC)
Security	Role-based access control, authentication, authorization
Redundancy	Controlled via normalization
Integrity Enforcement	Constraints (PRIMARY KEY, FOREIGN KEY, CHECK, NOT NULL)
Crash Recovery	Automatic — transaction logs, journaling, checkpoints

Common Examples

- **MySQL** — open-source relational DBMS, widely used in web applications
 - **Oracle Database** — enterprise-grade, highly robust
 - **PostgreSQL** — open-source, ACID-compliant, extensible
 - **SQLite** — embedded, serverless, used in mobile apps and local apps
-

2.3 Key Comparative Dimensions

Table 1: Conceptual Differences

Aspect	File System	Database System
Data Storage	Files	Tables (Relations)
Data Organization	Unstructured / Semi-structured	Structured (schema-enforced)
Data Control	Application-dependent	DBMS-controlled
Data Access	Manual (custom code)	Query-based (SQL)

Table 2: Feature-Level Comparison

Feature	File System	Database System
Redundancy	High — same data in multiple files	Low — normalization eliminates duplication
Security	Low — file-level OS permissions only	High — authentication, authorization, RBAC
Consistency	Poor — updating one file leaves others stale	Good — changes reflected atomically across all
Scalability	Limited — performance degrades with data growth	High — optimized indexing, partitioning
Query Support	No — custom code needed	Yes — SQL
Transaction Support	None	Full — ACID properties
Crash Recovery	None — manual backup only	Supported — journaling, WAL, checkpoints
Cost	Less expensive	Higher cost (software + administration)

Table 3: Detailed DBMS vs File System

File System	DBMS
Manages & organises files on hard disk	Software to store and retrieve user's data
Redundant data is present	No presence of redundant data
Query processing is not efficient	Query processing is efficient
Data consistency is low	High consistency via normalisation
Less complex; no complicated transactions	More complex; handles complicated transactions easily
Less security	Supports more security mechanisms
Less expensive	Higher cost than File System
Does not support crash recovery	Crash recovery highly supported

3. Basic Definitions & Terminology

Core Definitions (from Elmasri & Navathe / Silberschatz)

Database

A **database** is a collection of **related data** — known facts that can be recorded and that have an **implicit meaning** within a specific application domain (the *mini-world*).

- **Properties of a database:**
 - Represents some aspect of the real world (the *mini-world* or *Universe of Discourse — UoD*)
 - A logically coherent collection of data with inherent meaning
 - Designed, built, and populated for a **specific purpose** and audience

Data

Data consists of known facts that can be recorded and have an implicit meaning.

- Raw, unprocessed facts (e.g., a student's name, a product's price)
- Becomes **information** when organized and interpreted in context

Database Management System (DBMS)

A **DBMS** is a software package/system to **facilitate the creation and maintenance** of a computerized database. It enables users to define, construct, manipulate, and share databases among various users and applications.

- **Key capabilities:** definition, construction, manipulation, sharing, protection, maintenance

Database System

A **Database System** = DBMS software + the data itself. Sometimes application programs are also included.

Database System

- DBMS Software (e.g., MySQL Engine)
- Stored Database (actual data files)
- Stored Database Definition — Metadata (Data Dictionary)
- Application Programs / Queries (optional inclusion)

Relational Terminology

Informal Term	Formal Relational Term	Meaning
Table	Relation	A named set of rows and columns
Column	Attribute / Domain	A named property of the relation
Row	Tuple	A single record / instance
Values in a column	Domain	The set of permissible values for an attribute
Table Definition	Relation Scheme	The structure/schema of the relation
Populated Table	Relation Instance / Extension	The current set of tuples in the relation

Example: STUDENT Relation

Relation Name: STUDENT

Attributes: Name, SSN, HomePhone, Address, OfficePhone, Age, GPA

Tuple example:

| Benjamin Bayer | 305-61-2435 | 373-1616 | 2918 Bluebonnet Lane | null | 19 | 3.21 |

Example: EMPLOYEE Relation Schema

EMPLOYEE

- |— ID → Primary Key Attribute (underlined)
 - |— ENAME → Non-primary key attribute
 - |— SALARY → Non-primary key attribute
 - |— DEPT → Non-primary key attribute
-

Prepared by Francis Reuben R

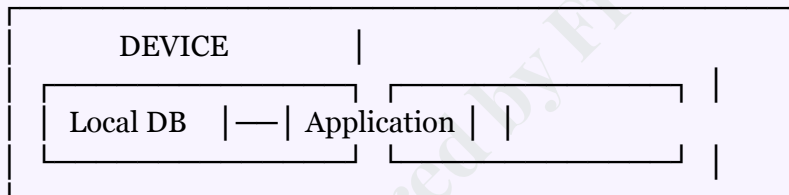
4. Database System Architecture

Database System Architecture describes the **structure of a DBMS** and explains how data is stored, accessed, and managed at different levels of abstraction.

Objectives of Database Architecture

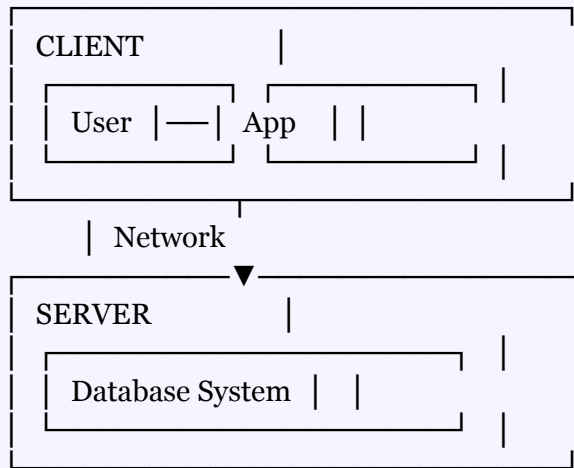
Objective	Purpose
Data Abstraction	Hide physical complexity from users
Data Independence	Change one level without affecting others
Security and Integrity	Protect data from unauthorized access and corruption
Efficient Data Access	Optimize query execution and storage I/O
Easy Maintenance	Simplify schema changes and administration

4.1 One-Tier Architecture



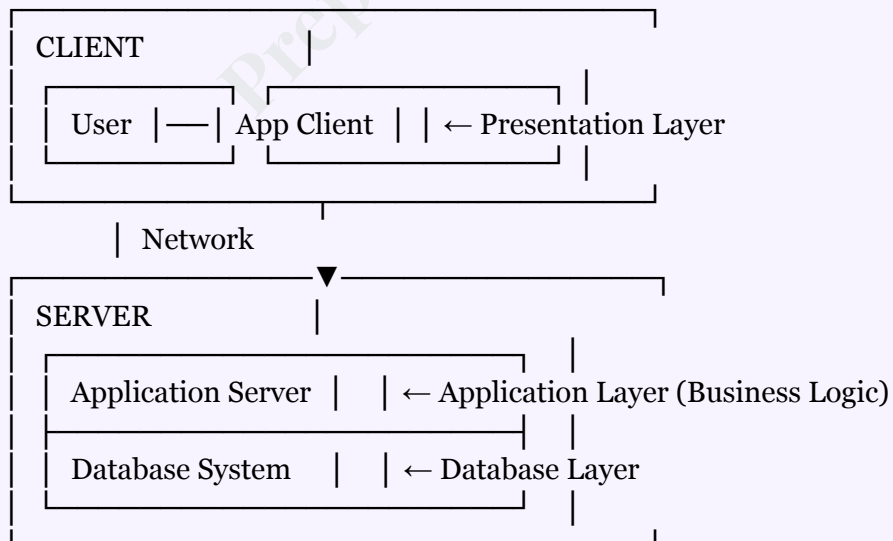
Property	Value
Description	Database, DBMS, and application reside on the same machine
User Interaction	User directly interacts with the database
Network Requirement	None
Complexity	Very simple
Use Case	Learning environments, personal apps, prototyping
Examples	MS Access, SQLite

4.2 Two-Tier Architecture



Property	Value
Description	Client communicates directly with the database server
Application Logic	Resides on client side
Database	On server side
Example	Desktop application (e.g., Java Swing app) connected to MySQL
Limitations	Less secure; business logic exposed on client; not suitable for large-scale deployments

4.3 Three-Tier Architecture



Layer	Name	Responsibility
Layer 1	Presentation Layer	User interface (browser, mobile app, desktop GUI)
Layer 2	Application Layer	Business logic, processing, rules enforcement
Layer 3	Database Layer	Data storage, retrieval, and management via DBMS

Property	Value
Usage	Most widely used architecture
Security	High — database not directly exposed to client
Scalability	Better — each layer scales independently
Maintenance	Easy — modify one layer without affecting others
Examples	Banking applications, E-commerce platforms (Amazon, Flipkart)

5. Three-Schema Architecture (ANSI/SPARC)

Proposed by the **ANSI/SPARC** committee (1975) to achieve **data abstraction** and **data independence** by separating a database system into three distinct levels.

External Level (User Views — View 1, View 2, View 3)

↑ ← Logical Data Independence

Conceptual Level (Logical Schema — tables, relationships, constraints)

↑ ← Physical Data Independence

Internal Level (Physical Schema — files, indexes, storage structures)

↑

DataStore (Actual physical storage)

Visual Reference: [Three-Level Architecture of DBMS — GeeksforGeeks](#)

5.1 External Level

The **External Level** (also called the *View Level* or *External Schema*) is the **highest level of abstraction** — it represents the **user's view** of the database.

- Different users (or user groups) see **different subsets** of the database through **views**.
- Hides all data that is irrelevant or unauthorized for a specific user.
- Each external schema is a **partial description** of the database tailored to a specific user role.

Key Properties:

- Multiple external schemas can exist for a single database
- No user sees the entire conceptual schema unless explicitly granted
- Provides **security** by exposing only necessary data

Example:

Same University Database:

Student View → sees: StudentID, Name, Marks, Attendance

Admin View → sees: All student records, fees, personal details

Faculty View → sees: Course enrollments, marks entry interface

5.2 Conceptual Level

The **Conceptual Level** (also called the *Logical Level* or *Conceptual Schema*) describes the **logical structure of the entire database** — what data is stored and the relationships among entities.

- Defines **tables, attributes, data types, relationships, constraints, and semantic information**.
- **Independent of physical storage** — does not describe how or where data is stored.
- Maintained by the **Database Administrator (DBA)**.
- Serves as the **mediating layer** between external views and internal storage.

What the Conceptual Schema Defines:

- Entity types and their attributes
- Relationships between entity types
- Integrity constraints (primary keys, foreign keys, CHECK constraints)
- Security and authorization information (at logical level)

Example (from slides):

```
-- Conceptual schema definition
CREATE TABLE Student (
  Stu_ID INT PRIMARY KEY,
  Stu_Name VARCHAR(100) NOT NULL,
  Proj_ID INT REFERENCES Project(ID)
);
```

5.3 Internal Level

The **Internal Level** (also called the *Physical Level* or *Internal Schema*) describes the **physical storage structure** of the database on storage media.

- Describes how data is **physically stored**: file organization, record formats, index structures, access paths, memory allocation.
- Concerned with **performance optimization**.
- Managed by the DBMS in collaboration with the OS storage manager.

What the Internal Schema Specifies:

- Storage record formats
- Ordering of stored records
- Indexes (B-trees, hash indexes)
- Encryption of stored data
- Data compression techniques
- Physical block/page sizes

Example:

Internal Schema for Student table:

- Stored as B+ tree indexed on Stu_ID
 - Pages of 8KB, records packed contiguously
 - Secondary index on Stu_Name (hash)
 - Stored in /var/lib/mysql/university/student.ibd
-

6. Data Model, Schema, and Instances

Data Model

A **Data Model** is a collection of concepts used to describe the structure of a database — including the **data types**, **relationships**, and **constraints** that apply to the data.

Types of Data Models:

Model	Description	Example
Relational Model	Data as 2D tables (relations)	MySQL, PostgreSQL
Hierarchical Model	Data as tree structures	IBM IMS
Network Model	Data as graph (records + sets)	IDMS
Object-Oriented Model	Data as objects with attributes and methods	ObjectDB
Entity-Relationship (ER) Model	High-level conceptual model using entities and relationships	Used in DB design phase
Document Model	Data as JSON/BSON documents	MongoDB

Database Schema

Database Schema = the **logical design** of the database. It is the blueprint that defines how data in a database is structured and how elements like foreign keys, primary keys, data types, and fields relate to each other.

- Defined using **coded rules** (DDL statements)
- The schema is **relatively static** — it changes infrequently (only during structural modifications)
- Analogous to the **type definition** in a programming language

Schema components include:

- Table names and column definitions
- Data types for each column
- Primary key designations
- Foreign key relationships

- Integrity constraints (CHECK, UNIQUE, NOT NULL)
- Index definitions

Example — Relational Schema (E-Commerce):

Customer (CustomerID PK, FirstName, LastName, Email)

Order (OrderID PK, CustomerID FK, OrderDate)

OrderDetail (OrderID PK FK, ProductID PK FK, Quantity)

Product (ProductID PK, ProductName, Price, CategoryID FK)

Category (CategoryID PK)

Database Instance

Database Instance = a **snapshot** of the data in the database at a **specific point in time**. Also called the *database state* or *extension* of the schema.

- Unlike the schema (which is static), the instance **changes with every INSERT, UPDATE, or DELETE** operation.
- At any point in time, the database has exactly one *current instance*.

Analogy:

Concept	Programming Analogy
Schema	Class / Type definition
Instance	Object / Variable holding actual values

Example:

Schema: Student(Stu_ID INT, Stu_Name VARCHAR(100), GPA DECIMAL(3,2))

Instance at t=Monday:

(1, 'Alice', 3.8)

(2, 'Bob', 3.5)

Instance at t=Tuesday (after INSERT):

(1, 'Alice', 3.8)

(2, 'Bob', 3.5)

(3, 'Charlie', 3.9) ← new tuple added

7. Data Independence

Data Independence is the ability to **modify the schema at one level** of the three-schema architecture **without having to alter the schema at the next higher level** — i.e., changes in lower levels do not cascade upward.

This is one of the most critical properties of a well-designed DBMS.

Two Types of Data Independence

External Level

↑ ← LOGICAL DATA INDEPENDENCE

Conceptual Level

↑ ← PHYSICAL DATA INDEPENDENCE

Internal Level

7.1 Physical Data Independence

Physical Data Independence is the ability to **modify the physical/internal schema** without requiring changes to the **conceptual/logical schema** or the **external schemas** (user views).

- Changes to storage structures, indexing strategies, or physical file organization do **not** affect the logical model or application programs.
- Achieved by maintaining the **conceptual-internal mapping** in the DBMS.

Impact Diagram:

View / External Level ← NOT AFFECTED

Logical / Conceptual Level ← NOT AFFECTED

Physical / Internal Level ← CHANGES MADE HERE

Why it matters:

- Allows DBAs to **optimize physical storage** (change from sequential files to B-tree indexes) without modifying the application code.
- Enables **database tuning** and **hardware migration** transparently.

Example:

Scenario: DBA moves the Student table from an HDD to an SSD and changes the index from sequential scan to B+ tree.

Effect on application:

```
SELECT * FROM Student WHERE Stu_ID = 5;
```

→ Query still works identically. Zero application change needed.

7.2 Logical Data Independence

Logical Data Independence is the ability to **modify the conceptual/logical schema** without requiring changes to the **external schemas** (user views) or the **application programs**.

- Changes to the logical structure of the database (adding new tables, adding columns, splitting/merging tables) do not break existing user views or applications.
- **Harder to achieve** than physical independence because changes to the logical schema more directly affect what users see.
- Achieved through **view definitions** and **conceptual-external mappings**.

Impact Diagram:

View / External Level ← NOT AFFECTED

Logical / Conceptual Level ← CHANGES MADE HERE

Physical / Internal Level ← NOT AFFECTED

Why it matters:

- Ensures existing applications don't break when the schema evolves (e.g., when adding a new **Genre** column to a **Books** table).
- Key enabler for **database evolution** without system-wide rewrites.

Example (Library System):

Initial Schema: Books(Title, Author, PublicationYear)

Change: ADD COLUMN Genre VARCHAR(50) to Books

Effect on applications:

- Staff software still works (sees Genre in catalog, displays it)
 - Reader website still works (search still functions as before)
 - Neither application required rewriting
-

Comparison: Physical vs. Logical Data Independence

Property	Physical Independence	Logical Independence
What changes	Internal/Physical schema	Conceptual/Logical schema
What is protected	Conceptual & External schemas	External schemas only
Difficulty	Easier to achieve	Harder to achieve
Mapping involved	Conceptual-Internal mapping	External-Conceptual mapping
Typical use case	Storage optimization, hardware migration	Schema evolution, adding attributes

8. Database Languages and Interfaces

A DBMS provides multiple **sub-languages** for different aspects of database interaction. Together they form the complete **Data Sublanguage**.

Overview

Language	Full Form	Purpose	Who Uses It
DDL	Data Definition Language	Define/modify schema structure	DBA, Developers
DML	Data Manipulation Language	Read and modify data	Developers, Analysts, End Users
DCL	Data Control Language	Manage access permissions	DBA, Security Teams
TCL	Transaction Control Language	Manage transaction boundaries	DBA, Developers, Data Engineers

8.1 DDL — Data Definition Language

DDL commands allow users to **define, alter, and manage** the database schema (structure) in SQL. They operate on the **metadata** (data dictionary) of the database, not the data itself.

- DDL statements are **auto-committed** in most RDBMS — they cannot be rolled back.
- When DDL is compiled, the resulting schema definitions are stored in the **Data Dictionary** (system catalog).

DDL Commands

Command	Description	Syntax Pattern
CREATE	Create a new database object (table, view, index, schema)	CREATE TABLE tablename (...)
ALTER	Modify an existing object (add/drop/rename columns, change data types)	ALTER TABLE tablename ADD COLUMN ...
DROP	Permanently delete a table/database and all its data	DROP TABLE tablename
TRUNCATE	Remove all rows from a table without deleting the table structure	TRUNCATE TABLE tablename
RENAME	Change the name of a table	RENAME TABLE old TO new
COMMENT	Add documentation to SQL code	-- single line / /* multi-line */

Example — Define a Database Schema

-- Step 1: Create the database

```
CREATE DATABASE CustomerDB;
```

```
USE CustomerDB;
```

-- Step 2: Define the Customers table with data types and constraints

```
CREATE TABLE Customers (  
    customer_id    INTEGER    PRIMARY KEY,  
    customer_name  VARCHAR(100) NOT NULL,  
    date_of_birth  DATE       NOT NULL,  
    account_balance DECIMAL(10, 2) CHECK (account_balance >= 0)  
);
```

-- Constraints applied:

-- PRIMARY KEY: uniquely identifies each customer

-- NOT NULL: name and DOB must always be provided

-- CHECK: balance cannot be negative

-- Step 3: Alter table to add a new column (Logical Data Independence in action)

ALTER TABLE Customers ADD COLUMN email VARCHAR(255);

-- Step 4: Rename a column

ALTER TABLE Customers RENAME COLUMN customer_name TO full_name;

8.2 DML — Data Manipulation Language

DML is a set of SQL commands used to **access, insert, modify, and delete data** within existing database tables or query views. It operates on the **database instance** (actual data rows).

- DML statements are **NOT auto-committed** — they participate in transactions and can be rolled back.
- Two types of DML:
 - **Procedural DML:** Specifies *what* data is needed *and how* to get it (e.g., relational algebra)
 - **Declarative (Non-procedural) DML:** Specifies *what* data is needed without specifying *how* (SQL **SELECT**)

DML Commands

Command	Type	Description
SELECT	Access (Read)	Retrieve rows from one or more tables
INSERT	Change	Add new rows to an existing table
UPDATE	Change	Modify values in existing rows
DELETE	Change	Remove rows from an existing table

Example — Construct and Manipulate Database

-- INSERT: Load initial data

```
INSERT INTO Customers (customer_id, customer_name, date_of_birth, account_balance)
VALUES
```

```
(1, 'John Doe', '1985-05-15', 1000.00),
(2, 'Jane Smith', '1990-07-22', 1500.50),
(3, 'Alice Johnson', '1988-11-03', 1200.75);
```

-- SELECT: Retrieve (Querying)

```
SELECT customer_name, account_balance
FROM Customers
```

```
WHERE account_balance > 1100.00;
```

-- Returns: Jane Smith (1500.50), Alice Johnson (1200.75)

-- UPDATE: Modification

```
UPDATE Customers
SET account_balance = account_balance + 500.00
WHERE customer_id = 1;
```

-- DELETE: Deletion

```
DELETE FROM Customers
WHERE customer_id = 3;
```

8.3 DCL – Data Control Language

DCL consists of SQL statements that **control security and concurrent access** to database objects by managing user privileges.

- DBAs and security teams use DCL to enforce **who can perform what operations** on which database objects.
- Ensures sensitive data is protected by granting permissions only to authorized users.
- DCL statements are typically **auto-committed**.

DCL Commands

Command	Description	Example
GRANT	Give specific privileges to a user or role	GRANT SELECT, INSERT ON Customers TO analyst_user;
REVOKE	Remove previously granted privileges	REVOKE INSERT ON Customers FROM analyst_user;

Privileges That Can Be Granted

Privilege	Meaning
SELECT	Read data from a table
INSERT	Add new rows
UPDATE	Modify existing rows
DELETE	Remove rows
ALTER	Modify table structure
ALL PRIVILEGES	All of the above

Example

```
-- Create a read-only analyst role
CREATE USER 'data_analyst'@'localhost' IDENTIFIED BY 'securePass123';
GRANT SELECT ON CustomerDB.Customers TO 'data_analyst'@'localhost';
-- Later, revoke update access from a contractor
REVOKE UPDATE ON CustomerDB.Customers FROM 'contractor_user'@'localhost';
```

8.4 TCL — Transaction Control Language

TCL consists of SQL statements that **manage transaction boundaries and integrity** — ensuring that sequences of DML operations are treated as atomic units.

- A **transaction** is a logical unit of work that must be completed entirely or not at all (ACID guarantee).
- TCL enables DBAs and developers to **commit**, **roll back**, or **savepoint** changes.

ACID Properties (Foundation of TCL)

Property	Meaning
A — Atomicity	All operations in a transaction succeed or all are rolled back
C — Consistency	Transaction takes DB from one valid state to another
I — Isolation	Concurrent transactions do not interfere with each other
D — Durability	Committed changes persist even after system failure

TCL Commands

Command	Description
COMMIT	Permanently save all changes made in the current transaction
ROLLBACK	Undo all changes made since the last COMMIT or SAVEPOINT
SAVEPOINT	Set a named intermediate point within a transaction
RELEASE SAVEPOINT	Remove a savepoint (cannot rollback to it after this)
SET TRANSACTION	Configure transaction properties (isolation level, read/write mode)

Example

```
BEGIN; -- Start transaction
```

```
UPDATE Accounts SET balance = balance - 500 WHERE account_id = 101; -- Debit
```

```
UPDATE Accounts SET balance = balance + 500 WHERE account_id = 202; -- Credit
```

```

-- If both succeed:
COMMIT; -- Make permanent

-- If any error occurs:
ROLLBACK; -- Undo both operations — atomicity preserved

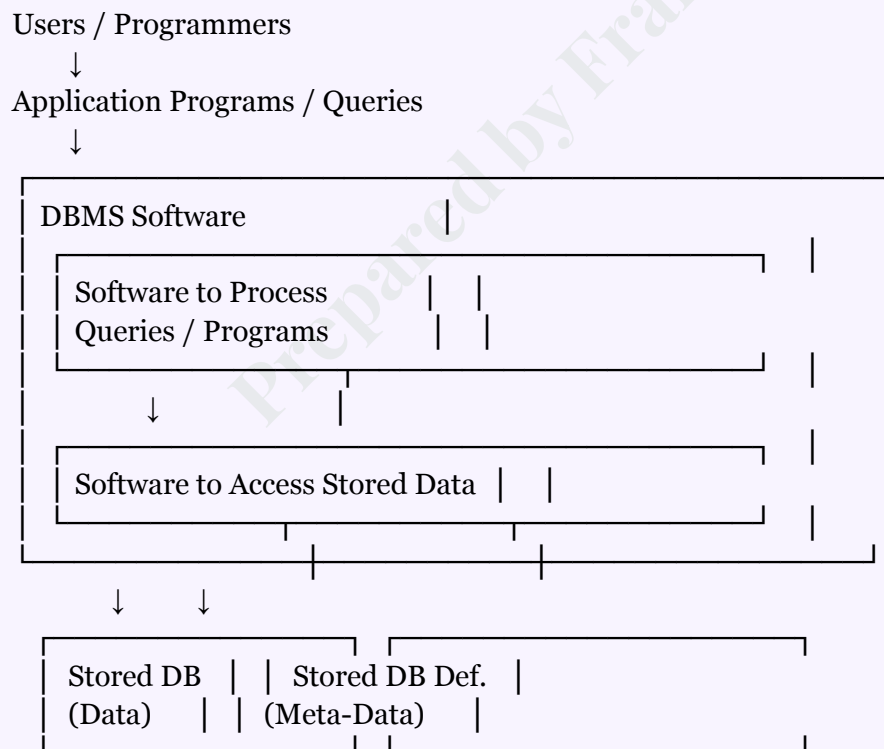
-- Using SAVEPOINT:
BEGIN;
INSERT INTO Orders VALUES (5001, 101, '2025-01-01');
SAVEPOINT after_insert;

UPDATE Inventory SET quantity = quantity - 1 WHERE product_id = 99;
-- If this UPDATE fails:
ROLLBACK TO after_insert; -- Undo only the UPDATE, keep the INSERT
COMMIT;

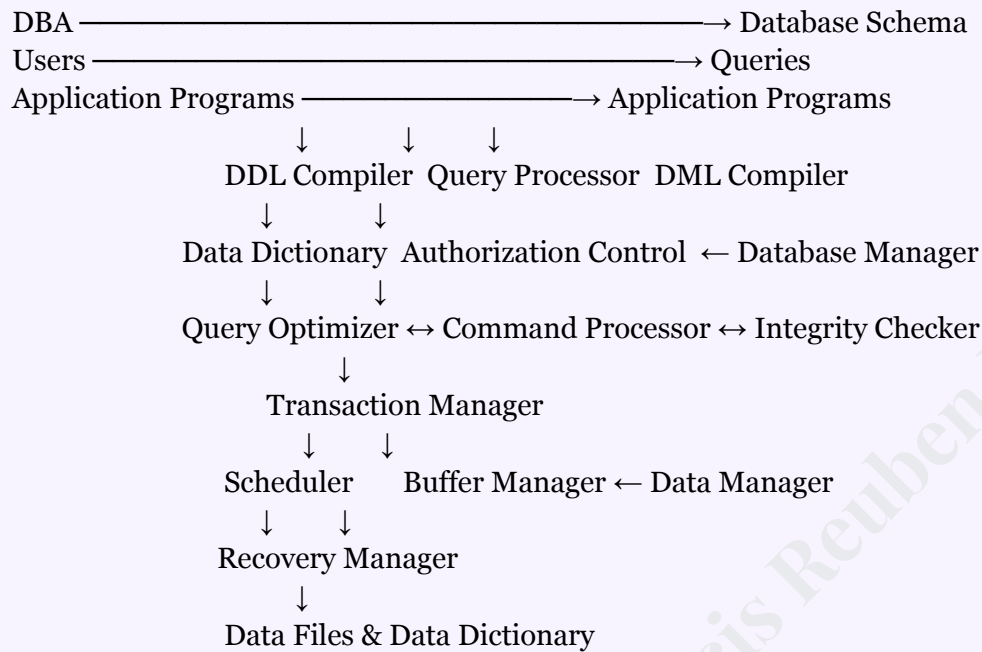
```

9. Overall Database Structure

Simplified Database Environment Diagram



Full DBMS Component Architecture:



9.1 Storage Components

Data storage in a DBMS is classified into three hierarchical categories based on **data persistence** and **speed of access**.

Speed ↑	Capacity ↓
Cache (Primary)	
Main Memory / RAM (Primary)	
Flash Memory (Secondary)	
Magnetic Disk (Secondary)	
Optical Disk (Tertiary)	
Magnetic Tapes (Tertiary)	
Speed ↓	Capacity ↑

Tier 1: Primary Storage

Primary Storage = storage directly accessible by the CPU for active computation.
Volatile (data lost on power off).

Component	Description	Characteristics
CPU Cache (L1/L2/L3)	Fastest memory, on/near CPU chip	Smallest capacity (KB–MB); nanosecond access
Main Memory (RAM)	Active working memory	Fast (ns– μ s); volatile; expensive per GB

- DBMS loads data pages from disk into RAM (the **buffer pool**) before processing.
- **Buffer Manager** controls which pages reside in RAM at any time.

Tier 2: Secondary Storage

Secondary Storage = persistent, non-volatile storage directly attached to the computer. The primary residence of database files.

Component	Description	Characteristics
Flash Memory (SSD)	Solid-state non-volatile storage	Fast random I/O (μ s); lower latency than HDD
Magnetic Disk (HDD)	Rotating platters, magnetic read/write	Slower (ms); large capacity; cost-effective

- **Primary location** where database data files, index files, and transaction logs are stored.
- DBMS uses **page/block I/O** to transfer data between secondary and primary storage.

Tier 3: Tertiary Storage

Tertiary Storage = offline or near-line storage for archival and backup purposes.

Component	Description	Characteristics
Optical Disks (CD/DVD/Blu-ray)	Laser-read optical media	Moderate capacity; used for archival

Magnetic Tapes

Sequential-access
magnetic media

Very large capacity; very slow; lowest cost
per GB; used for long-term backup

Storage Hierarchy — Complete Summary

Level	Type	Speed	Capacity	Volatility	Cost/GB
L1/L2 Cache	Primary	~1 ns	KB–MB	Volatile	Very High
RAM	Primary	~10–100 ns	GB	Volatile	High
SSD (Flash)	Secondary	~0.1 ms	GB–TB	Non-volatile	Medium
HDD	Secondary	~5–10 ms	GB–TB	Non-volatile	Low
Optical Disk	Tertiary	~100 ms	GB	Non-volatile	Very Low
Magnetic Tape	Tertiary	Seconds	TB–PB	Non-volatile	Lowest

9.2 Metadata

Metadata = "data about data." It describes the **structure, meaning, and constraints** of the user data stored in the database.

What Metadata Records:

- Table names, column names, data types, lengths
- Primary key designations
- Foreign key relationships
- Index definitions
- Constraint definitions
- User privileges and roles
- Storage parameters

Where Metadata is Stored:

- In the **System Catalog** (also called the **Data Dictionary**)
- Accessible by the DBMS internally and by the system administrator
- In MySQL: `information_schema` database; In PostgreSQL: `pg_catalog`

Metadata vs. Data:

Data → The actual content: ('John Doe', 3.8, 'CS')

Metadata → The description: (column: 'name', type: VARCHAR(100), NOT NULL)

Granularity Levels of Metadata:

Collections of Collections → Database-level metadata (DB name, character set)

Collections → Table-level metadata (table name, engine type)

Items → Column-level metadata (column name, data type, constraints)

Accessing Metadata in MySQL:

-- View all tables in a database

```
SELECT table_name, table_type, engine
```

```
FROM information_schema.TABLES
```

```
WHERE table_schema = 'CustomerDB';
```

-- View column metadata

```
SELECT column_name, data_type, is_nullable, column_key
```

```
FROM information_schema.COLUMNS
```

```
WHERE table_name = 'Customers';
```

9.3 Users in a DBMS Environment

User Type	Role	Interaction Method
Database Administrator (DBA)	Defines schema, manages access, optimizes performance, backup/recovery	DDL, system tools
Database Designers	Design the conceptual and logical schema (ER diagrams, normalization)	CASE tools, DDL
Application Programmers	Write programs that access DB via DML	Embedded SQL, APIs (JDBC, ODBC)
End Users (Casual)	Query DB ad hoc using SQL or BI tools	SQL clients, BI dashboards
End Users (Naive/Parametric)	Use pre-built application interfaces	Web apps, mobile apps
System Analysts	Define requirements for database applications	Requirements docs

9.4 Database Backup and Recovery Mechanisms

Why Backup?

Backing up data is **vital for business continuity** — lost information can cause major crises or outright business failure.

Common Failure Types

Failure Type	Description	Example
Transaction Failure	Error within a single transaction (logic error, constraint violation)	Division by zero in a stored procedure
System Outage	Power failure, OS crash, hardware failure	Server power cut mid-transaction
Media Failure	Physical disk becomes unreadable/corrupted	HDD head crash
Disaster	Physical destruction of the database facility	Fire, flood, earthquake

Recovery Facilities

Facility	Description
Backup Facilities	Periodic full/incremental/differential copies of the entire database stored offline
Journalizing Facilities (Write-Ahead Log — WAL)	Continuous log of all changes (before and after images of every modified page)
Checkpoint Facility	Periodic synchronization point — all in-memory pages are flushed to disk; reduces recovery time
Recovery Manager	DBMS component responsible for restoring the database to a consistent state after failure

Recovery Techniques

Technique	Description	Use Case
Undo (Rollback)	Reverse changes of incomplete/failed transactions	Transaction failure
Redo (Roll forward)	Reapply committed changes not yet written to disk	System failure after COMMIT but before flush

Deferred Update	Changes written to log only; applied to DB only at COMMIT	Conservative approach
Immediate Update	Changes applied to DB as they occur, with log entries for undo	Aggressive performance approach

Responsibility

- Database recovery is primarily the responsibility of the **Database Administrator (DBA)** with the assistance of the DBMS's built-in **Recovery Manager**.

Example — MySQL Backup and Recovery

Full backup using mysqldump

```
mysqldump -u root -p CustomerDB > customer_backup_2025.sql
```

Restore from backup

```
mysql -u root -p CustomerDB < customer_backup_2025.sql
```

Enable binary logging for point-in-time recovery (in my.cnf)

[mysqld]

log_bin = /var/log/mysql/mysql-bin.log

binlog_format = ROW

10. Typical DBMS Functionality

A complete DBMS must support the following core functions (per Elmasri & Navathe):

10.1 Define

Define the database in terms of **data types, structures, and constraints** using DDL.

-- Define with data types, NOT NULL, CHECK constraints

```
CREATE TABLE Customers (
```

```
    customer_id    INTEGER    PRIMARY KEY,
```

```
    customer_name  VARCHAR(100) NOT NULL,
```

```
    date_of_birth  DATE        NOT NULL,
```

```
    account_balance DECIMAL(10,2) CHECK (account_balance >= 0)
```

```
);
```

10.2 Construct / Load

Build the initial database by **loading data** onto secondary storage.

-- Load initial records

```
INSERT INTO Customers VALUES
(1, 'John Doe', '1985-05-15', 1000.00),
(2, 'Jane Smith', '1990-07-22', 1500.50),
(3, 'Alice Johnson', '1988-11-03', 1200.75);
```

10.3 Manipulate

Retrieve, modify, and report on data:

Operation	SQL Mechanism	Example
Retrieval (Querying)	SELECT + WHERE, JOIN, aggregation	SELECT * FROM Customers WHERE balance > 1000
Report Generation	Aggregation + GROUP BY + ORDER BY	Monthly sales summary
Modification — Insert	INSERT INTO ... VALUES ...	Add new customer
Modification — Update	UPDATE ... SET ... WHERE ...	Update balance
Modification — Delete	DELETE FROM ... WHERE ...	Remove inactive account
Web Application Access	Via application layer (JDBC, ODBC, ORM)	REST API querying DB

10.4 Process and Share

Support **concurrent multi-user access** while maintaining **data validity and consistency**.

- Implemented via **Concurrency Control** (locking, MVCC)
- Ensures transactions don't interfere with each other (Isolation)
- **Transaction Manager** and **Scheduler** coordinate concurrent operations

11. Relational Model — Core Terminology

The STUDENT Relation (Example from Slides)

Relation Name: STUDENT

Attributes: Name | SSN | HomePhone | Address | OfficePhone | Age | GPA

↑ These are the ATTRIBUTES (columns)

Tuples (rows):

Benjamin Bayer | 305-61-2435 | 373-1616 | 2918 Bluebonnet Lane | null | 19 | 3.21

Katherine Ashly | 381-62-1245 | 375-4409 | 125 Kirby Road | null | 18 | 2.89

Dick Davidson | 422-11-2320 | null | 3452 Elgin Road | 749-1253 | 25 | 3.53

Charles Cooper | 489-22-1100 | 376-9821 | 265 Lark Lane | 749-6492 | 28 | 3.93

Barbara Benson | 533-69-1238 | 839-8461 | 7384 Fontana Lane | null | 19 | 3.25

Formal Definitions

Relation

A **relation** $R(A_1, A_2, \dots, A_n)$ is defined on a schema R and consists of a set of **tuples** $\{t_1, t_2, \dots, t_n\}$ where each tuple t_i is an ordered list of values conforming to the attribute domains.

- **Cardinality** = number of tuples (rows) = m
- **Degree / Arity** = number of attributes (columns) = n

Attribute

An **attribute** A_i is a named column of a relation, associated with a **domain** $\text{dom}(A_i)$ — the set of atomic values that column may contain.

- **Example:** A_{Age} attribute with domain = positive integers 0–150

Tuple

A **tuple** t in a relation $R(A_1, \dots, A_n)$ is an ordered list of values $\langle v_1, v_2, \dots, v_n \rangle$ where each $v_i \in \text{dom}(A_i)$ or is **NULL**.

- Represents a **single entity instance** (e.g., one student, one employee)

Domain

A **domain** is the set of atomic (indivisible) values permitted for an attribute.

- $\text{dom}(\text{GPA}) = \{x \in \mathbb{R} \mid 0.0 \leq x \leq 4.0\}$
- $\text{dom}(\text{SSN}) = 9\text{-digit strings matching the format NNN-NN-NNNN}$

NULL Value

NULL represents an **unknown**, **inapplicable**, or **missing** value for an attribute in a tuple.

- $\text{NULL} \neq 0$ and $\text{NULL} \neq ""$ — it means the value is absent/unknown
- Any arithmetic or comparison with NULL yields NULL (three-valued logic: TRUE, FALSE, UNKNOWN)

Keys in the Relational Model

Key Type	Definition	Example
Super Key	Any set of attributes that uniquely identifies a tuple	$\{\text{SSN}\}$, $\{\text{SSN}, \text{Name}\}$, $\{\text{SSN}, \text{Age}\}$
Candidate Key	A <i>minimal</i> super key — no proper subset is also a super key	$\{\text{SSN}\}$ (minimal)
Primary Key (PK)	The candidate key chosen by the designer to uniquely identify tuples	SSN designated as PK
Foreign Key (FK)	An attribute in relation R that references the PK of relation S	CourseID in ENROLLMENT references CourseID in COURSE
Composite Key	A key consisting of two or more attributes	(OrderID, ProductID) in ORDER_DETAIL

Integrity Constraints

Constraint	Rule
Entity Integrity	No attribute of a primary key may be NULL
Referential Integrity	A FK value must either match an existing PK in the referenced relation, or be NULL
Domain Constraint	Every value must belong to the defined domain of its attribute
Key Constraint	Primary key values must be unique across all tuples

Example Database Instance (from slides)

COURSE Table:

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION Table:

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
135	CS3380	Fall	05	Stone

PREREQUISITE Table:

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

12. MCQ Drill — Recall & Revise

The following are the MCQs from the lecture slides with answers and explanations.

Q1. In database terminology, what is an attribute?

- A) A unique identifier for a record
- B) ☒ **A field or column in a table that holds data**
- C) A table that stores multiple records
- D) A row in a table

Explanation: An attribute is a named column in a relation, representing a property of the entity.

Q2. In database terminology, what is a relation?

- A) A unique identifier for a record
- B) A row in a table
- C) ☒ **A table in a database that consists of rows and columns**
- D) A field or column

Explanation: In the formal relational model, a relation = table.

Q3. In database terminology, what is a tuple?

- A) A unique identifier
- B) A table
- C) ☒ **A row in a table containing a set of values for each attribute**
- D) A field or column

Explanation: A tuple is an ordered list of values corresponding to one entity instance.

Q4. Which is NOT an advantage of a database system over a file system?

- A) Data redundancy and inconsistency reduction
- B) Data sharing and security
- C) ☒ **Faster direct file access**
- D) Better data integrity and consistency

Explanation: File systems may offer faster direct access to a single specific file, but this is not an advantage of database systems. DBs trade some raw speed for structure, consistency, and safety.

Q5. In a database system, what does "Schema" refer to?

- A) The actual data stored
- B) ☒ **The logical structure that defines how data is organized**
- C) Temporary storage for transactions
- D) A collection of primary keys

Explanation: Schema = the blueprint/design; Instance = the actual data at a point in time.

Q6. Which is an example of a DML command?

- A) CREATE
- B) ALTER
- C) ☒ **INSERT**
- D) GRANT

Explanation: CREATE and ALTER are DDL; GRANT is DCL; INSERT is DML (modifies data).

Q7. Which is NOT a storage component of a database system?

- A) Index files
- B) Metadata tables
- C) ☒ **CPU cache**
- D) Data files

Explanation: CPU cache is a hardware-level resource managed by the CPU/OS, not a database storage component per se. Index files, metadata tables, and data files are all explicit DBMS storage components.

Q8. What is the primary purpose of database backups and recovery mechanisms?

- A) To delete unnecessary records
 - B) To secure data from unauthorized users
 - C) ☒ **To restore data in case of system failures or corruption**
 - D) To improve query execution speed
-

Q9. Which best describes the purpose of Data Independence?

- A) ☒ **It allows changes in data storage without affecting application programs**
 - B) It ensures only one user can access the DB at a time
 - C) It restricts unauthorized access
 - D) It improves network speed
-

Q10. What is the primary role of a DBA?

- A) Writing application programs
 - B) ☒ **Managing database storage and security**
 - C) Executing queries only
 - D) Designing network protocols
-

Q11. Which is NOT a function of Metadata?

- A) ☒ **Storing actual data records**
- B) Describing data types and constraints
- C) Defining relationships between tables
- D) Helping the DBMS understand data structure

Explanation: Metadata describes structure — it does not contain actual user data (tuples). Actual records are stored in data files.

Q12. Which database language controls access to a database?

- A) DDL
 - B) DML
 - C) ☒ **DCL (Data Control Language)**
 - D) TCL
-

Q13. Which statement about a Primary Key is true?

- A) A PK can have duplicate values
- B) A table can have multiple PKs

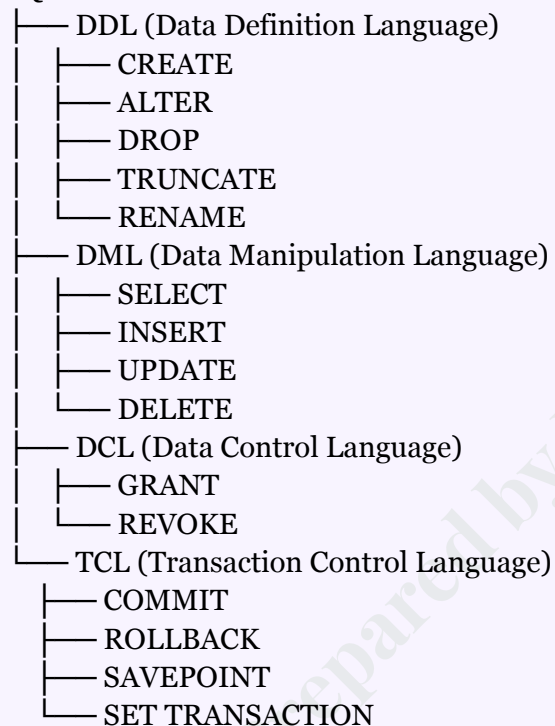
- C) ☒ A PK uniquely identifies each record in a table
D) A PK must always be numeric

Explanation: PKs enforce entity integrity — unique + NOT NULL. A table has exactly ONE primary key (though it may be composite).

Appendix: Key Formulas and Quick Reference

SQL Language Classification Summary

SQL



Three-Schema Architecture — Quick Reference

Level	Schema Name	Content	Independence
-------	-------------	---------	--------------

External	External Schema	User-specific views	↑ Logical
Conceptual	Conceptual Schema	Tables, relationships, constraints	↓ Physical
Internal	Internal Schema	Files, indexes, storage blocks	
Physical	Physical Storage	Actual bits on disk	

Data Independence — One-Line Definitions

Type	One-Line Definition
Physical	Change storage $\neq$ Change logical design
Logical	Change logical design $\neq$ Change user views

Architecture Comparison

Architecture	Tiers	Example
One-Tier	1 (DB + App on same machine)	MS Access, SQLite
Two-Tier	2 (Client + DB Server)	Desktop app + MySQL
Three-Tier	3 (UI + App Server + DB Server)	Web banking, E-commerce

End of DBMS Unit 1 Master Notes — RV University, CIE 1 Coverage: File Systems, DBMS Concepts, Architecture, Schema, Data Independence, DDL/DML/DCL/TCL, Storage, Metadata, Backup & Recovery